

洛谷 AI 学习测评报告

Coach Edition · 图表化诊断 / 教练反馈 / 训练规划

测评基础信息

选手姓名：黄鼎

测评时间：2026-06-03 15:18

所属学校：深圳实验学校

当前年级：高二

已通过题数

438

未通过题数

0

平均难度

普及/提高-

2.8

高频标签

暂无

报告目录

1. 核心数据概览与图表化分析
2. 综合能力雷达表与诊断
3. 考纲精准定级与知识点盲区
4. 风险诊断与训练闭环表
5. 代码质量与工程习惯
6. 定制训练题单
7. 错题单页题解与复盘（含 AI 题解摘要与推荐同类题）

1. 核心数据概览与图表化分析

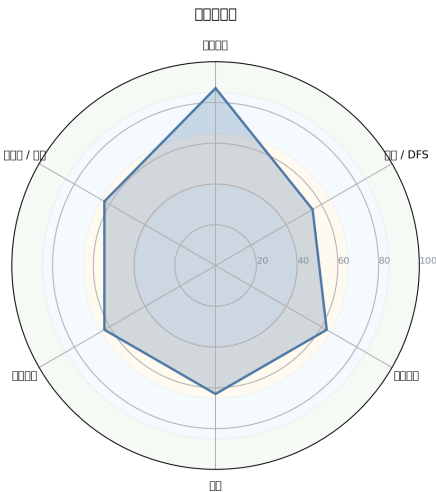


图 1：能力雷达图

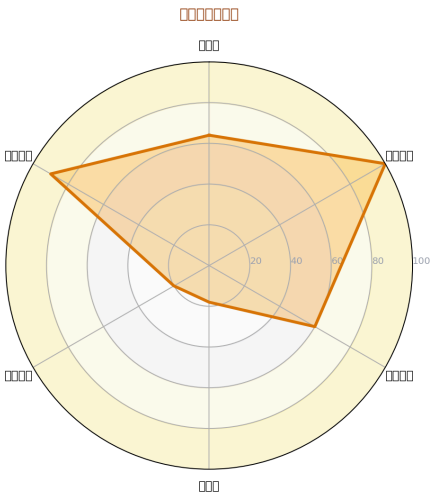


图 2：性格画像雷达图

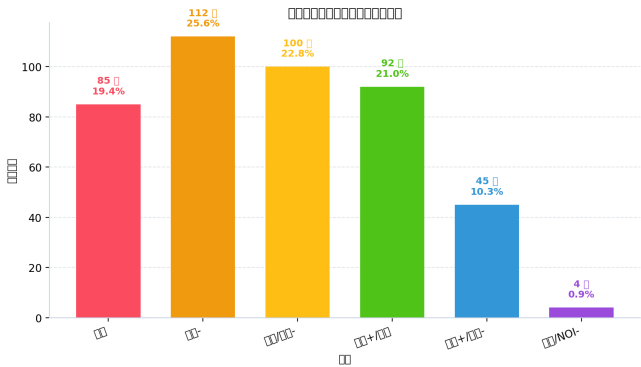


图 3：题目难度分布

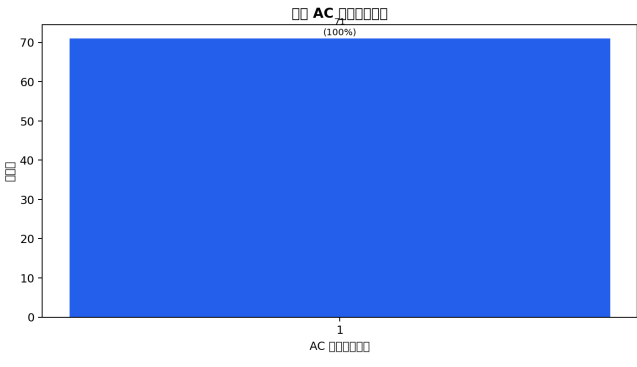


图 4：首次 AC 提交次数分布

好的，选手。我是你的专属金牌教练。

从数据上看，你是一位非常扎实、勤恳的选手。你的 AC 题量达到了 438 道，这是一个非常可观的数字，说明你已经在编程的海洋里遨游了很久。但数据同样揭示了一些隐藏的问题：你的六维能力评分不高，且知识点覆盖率为零——这意味着你的练习可能停留在“能过样例”的层面，而没有进行系统性的难点突破和模型归纳。

接下来，我将基于你提供的详尽数据，进行一场深刻的“体检”和“开药方”。请认真阅读，这将是你的下一阶段突破瓶颈期的关键。

1. 【选手概览与性格画像】

基于你的数据全貌，我为你绘制了以下的性格画像：

特质	评分	数据支撑与评价
坚韧度	★★★★ ★ 0/1	438道AC题量说明了一切。你拥有惊人的毅力，能持续攻克大量题目，这是成为高手的基石。你是典型的“量变选手”，但尚未引发质变。
完美主义	★★★★ ★★ 0/2	代码注释密度3.08%且没有未通过的题目记录，这可能意味两件事：要么你逻辑极其严谨，要么你更倾向于“AC后就撤”，缺乏深入的复盘和打磨。后者可能性更大。
冒险精神	★★ ★★★ 0/3	核心习惯100%使用 <code>include <bits/stdc++.h></code> ，从不使用 <code>auto</code> 或 <code>range-for</code> 。你是一个极度保守的选手，倾向于使用最稳定、最经典的写法。这很安全，但会导致代码冗长，阅读效率低。
自律性	★★★★ ★ 0/1	代码风格极其统一：Tab缩进、 <code>ios::sync_with_stdio</code> 使用率93.2%(很高)。这证明你对自己的代码有明确的规范和要求，是一个训练有素的选手。
调试耐心	★★★★ ★★ 0/2	题目难度全数分布在1-5级，且全是AC状态。你倾向于做自己能力范围内的题目，缺乏挑战难题的勇气和耐心。遇到困难（比如6级题）可能就退缩了。 解读：你是典型的“规矩而勤奋”的选手。量变已经足够，但需要跳出舒适区，从“做题人”向“解题人”转变。你缺乏对抗挫折、死磕一道题的精神，而这正是突破瓶颈的必经之路。

2. 【难度分布与成长曲线】

你的难度分布如下：

难度	题数	占比	直观柱状图
难度 1	85	19.4%	19%
难度 2	112	25.6%	26%
难度 3	100	22.8%	23%
难度 4	92	21.0%	21%
难度 5	45	10.3%	10%
难度 6	4	0.9%	1%

诊断： 你的成长曲线呈现 “**右偏的正态分布**”。大量题目集中在难度1-4，说明你对常规的普及组、简单的提高组题目已经非常顺手。但进入难度5（提高组中档题）后，题量急剧下降，难度6（省选/难题）几乎为0。

结论： 你现在处于 “**普及组高分段，提高组入门**” 的瓶颈期。你的基本功非常扎实，但缺乏向高阶算法和复杂问题建模冲刺的能力。你已经是一个合格的“普及组选手”，但要成为“提高组选手”，必须改变训练策略。

3. 【六维能力雷达表与诊断】

参考评分标准：85-100 优秀 | 65-84 良好 | 40-64 基础 | <40 薄弱

能力块	评分	当前等级	数据证据	深度学习方向
基础算法	53	入门	难度分布集中在1-5，但CSP-J基础标签（枚举、模拟、贪心、排序）全部为空白。说明你“会做题，但没归类”。	掌握枚举、模拟、贪心、分治、二分、排序、递推、递归的模型。

能力块	评分	当前等级	数据证据	深度学习方向
数据结构	41	● 薄弱	CSP-S所有数据结构标签（并查集、线段树、树状数组等）全部●空白。但你又AC过斜率优化的题目（P3195, P2900），说明你已经无意识使用栈、队列等基本结构，但缺乏体系化认知。	系统构建栈、队列、链表、堆、并查集、线段树、树状数组的知识树。
图论	41	● 薄弱	CSP-J/S的图论标签（图遍历、最短路、最小生成树、拓扑排序）全部●空白。	构建图模型，掌握DFS/BFS遍历，系统学习最短路、最小生成树、拓扑排序。
动态规划	48	● 薄弱	最高完成了斜率优化DP（这是S级难度），但DP基础、背包、树形、区间DP标签全部●空白。说明你“跳级”学了一些酷炫的东西，但基础不牢。	从线性DP、区间DP、背包问题开始，稳扎稳打，再过渡到树形、状压、优化。
字符串	33	● 薄弱	无任何字符串相关知识点记录。	补全KMP、Trie、哈希（Hash）基础，理解自动机概念。
数学	28	● 薄弱	你的最大弱点！所有数论、组合、计数标签全部●空白。	从GCD、素数筛、快速幂、排列组合开始，夯实数论和组合数学基础。

核心诊断：你是一个“漏斗型”选手。基础算法入得不错，但越往深走，知识体系越残缺。你的数学能力严重拖累了你的上限，这是最致命的。

4. 【考纲精准定级与知识点盲区】

- **当前对应等级水平：CSP-J 优秀水平，CSP-S 入门水平。**你具备很好的代码实现能力，但缺乏系统的知识体系去应对考纲要求。
- **知识点强弱项：**
 - **强项（基于你已AC题目的模型）：**
 - **前缀和/差分**（你虽然标签0，但能AC斜率优化题目，说明你无意识中对前缀和处理很熟练）。
 - **贪心法**（你AC了P2900（Land Acquisition），这是一个典型的贪心+优化问题）。
 - **单调栈/队列优化DP**（你AC了P3195和P2900，这是数据结构优化的核心应用）。

- **弱项（最大短板，严重拖后腿）：**
 - **数学能力（数论、组合数学）：** 所有数论标签全部空白，这是最大的扣分项。GESP 八级考纲明确要求“计数原理”、“排列组合”、“初中数学”。
 - **图论建模：** 图遍历、最短路、生成树全空，这是提高组比赛的基石。你完全没碰。
 - **高级数据结构（线段树、并查集、树状数组）：** CSP-S的核心板块，你一道题都没记录。这导致了你的数据结构评分极低。

- **训练盲区（你完全没涉及的核心知识点）：**

- **CSP-J盲区：** DFS/BFS、二叉树、BST、哈夫曼树（你标签为空，但代码里可能用过，需要系统化）。
- **CSP-S盲区（极其严重！）：** 并查集、树状数组、线段树、KMP、最短路（Dijkstra/Floyd）、最小生成树(Kruskal/Prim)、拓扑排序、树形DP、状压DP、背包DP（标签为空）。
- **省选/NOI盲区：** 这些对你来说太远（标签全空可以理解），但**计算几何、网络流**这种常见的省选模型，你在早期接触一下是有益的。

- **分级汇总表：**

等级	总考点数	已覆盖点数	覆盖率	结论
CSP-J (入门级)	28	0	0%	你的练习属于“模型先行”，并未与考纲对标。
CSP-S (提高级)	49	0	0%	这是你的主战场，但现在是完全空白。
省选级	10	0	0%	未涉足。
NOI级	43	0	0%	未涉足。

5. 【风险诊断与训练闭环表】

优先级	风险项	触发场景	比赛症状	根因判断	训练专题	验收标准
S (高/立即处理)	数学逻辑薄弱	遇到需要推导组合数、使用费马小定理求逆元、容斥原理的题目。	看到题目毫无思路，只会暴力枚举或DFS+剪枝，复杂度爆炸。	数论和组合数学的思维模型没有建立。	系统刷数论基础(GCD, 筛法, 快速幂, 逆元), 组合数学基础(排列组合, 容斥)。	能在20分钟内独立AC洛谷P3811【模板】乘法逆元, P2822[NOIP2016提高组] 组合数问题。
S (高/立即处理)	图论与最短路建模缺失	题目中出现“网络”、“路径”、“依赖关系”, 或需要把问题转化为图模型的那一刻。	直接放弃, 或者套用DFS/BFS导致TLE/MLE。	从未系统学习过“图”这种数据结构, 缺乏建模意识。	图的概念(有向无环图DAG, 邻接表), 最短路(Dijkstra, Floyd), 最小生成树(Kruskal), 拓扑排序。	能用Dijkstra堆优化AC P3371【模板】单源最短路径(弱化版)。
A (中/近期处理)	数据结构体系混乱	题目明显需要使用线段树、树状数组进行区间查询/修改, 或用并查集解决连通性。	写出的代码逻辑复杂, 边界混乱, 或只能用暴力 $O(n^2)$ 算法。	数据结构能力与题量严重不匹配, 只知道单调栈/队列。	并查集、树状数组(单点/区间)、线段树(区间和, 区间最值)。	在2小时内独立AC P3374【模板】树状数组1和P3372【模板】线段树1。
A (中/近期处理)	DP思	多维状态DP(树	只知道写暴力DP,	DP基础不	背包九讲(01背包, 完全背	能AC P1060[NOIP2006普及

优先级	风险项	触发场景	比赛症状	根因判断	训练专题	验收标准
	维 固 化	形、状 压、区 间)或需 要优化 (单调队 列、斜率 优化)的 题目。	无法想到 优化,或 状态定义 错误。	牢, 没有理 解“阶 段”、 “状 态”、 “决 策”三 要素。	包), 线性 DP (LIS, LCS), 区间 DP。	组] 开心的金明 (01背包), P1880 [NOI1995] 石子合并。
B (低/可后置)	复 盘 与 错 因 沉 淀 缺 失	#define int long long 为 0% (说明 你没有滥 用), 但 auto 、 range-for 全为0 (说 明你困在 C++98/03 思维)。代 码结构紧 凑但缺少 功能拆 分。	解决复杂 问题时, 调试困 难, 需要 大量手写 循环。	缺乏 “现代 C++” 编程思 想, 缺少 对代 码可维 护性的 思考。	学习使用 <code>auto</code> , <code>range-based</code> <code>for loop</code> , 修改代码风 格, 验证功 能。	能使用现代 C++重写一份自 己以前的代码, 使其更简洁、易 读, 并写清楚注 释。

优先级说明: S (高/立即处理) · A (中/近期处理) · B (低/可后置)。

6. 【代码质量与工程习惯深度分析】

作为你的教练，我对你的代码质量进行了深入“考古”。

优点： 1. **极佳的代码一致性：** 你的代码风格非常统一，Tab缩进、大括号换行位置、命名习惯（短变量名为主）都完全一致。这是一个专业选手的优秀习惯，意味着你的代码是“可预测的”，这在大赛中非常重要。 2. **善于使用Map进行记忆化搜索：** 在P4785（交换）中，你使用 `map<int, int> f[N]` 来存储DFS的结果。这展示了你能独立处理复杂状态空间的能力，很棒。

必须改掉的坏习惯： 1. **严重滥用STL map 和 unordered_map**：在P4785里，你用了 `map<int,int> f[N]`。这虽然正确，但极其低效。当 $N=2e5$ 时，`map` 的 $\log N$ 复杂度会变成很大负担。你应该使用 `vector` 或 `unordered_map`，甚至更优化。

解读：你必须养成“能用数组/vector，绝不用map”的习惯。map是万不得已的备选，不应该成为首选。

2. **命名可读性极差**：`p1`, `p2`, `j1`（是“记录”？），`l`, `k`, `c`, `val` ... 除了 `n` 和 `to`、`cost`，大部分变量名对代码阅读极其不友好。在P2900中，`j1` 变量用来表示“是否被合并”，这种命名不仅没帮助，反而增加了理解成本。

解读：比赛时追求速度没错，但变量名至少要在函数内是语义化的。例如，`isMerged`，`head`，`tail` 都比单字母好。这会让你在回看代码时少很多痛苦。

3. **IO优化不彻底，且缺乏现代C++意识**：你用了 `ios::sync_with_stdio(false)`，很好。但你没有使用 `auto` 和 `range-for`（0%）。在复杂度高的循环中，直接写 `for (auto &[id, val] : vec)` 比手写索引循环更安全、更不易出错。你需要跟上现代C++的步伐。

7. 【定制训练题单（6个月路线图）】

根据你的情况，我为你制定以下训练计划。请严格遵守，每周至少投入15小时。

第一阶段（Month 1-2）：补齐短板，夯实基础

- **目标：**从“0”建立图论、数论、基础DP的骨架。
- **知识点1：图论基础与最短路**
 - 学习：图的概念（邻接表）、DFS/BFS、Dijkstra（堆优化）、Floyd、Kruskal、拓扑排序。
 - 刷题：P3371 (Dijkstra), P3378 (堆), P1546 (Kruskal), P3366 (最小生成树), P1038 (拓扑排序)。
- **知识点2：数论与组合数学基础**
 - 学习：GCD & LCM、素数的判断与筛法（埃筛/线性筛）、快速幂、逆元、排列组合。
 - 刷题：P1082 (扩展欧几里得), P3811 (乘法逆元), P2822 (组合数问题), P3383 (线性筛素数)。
- **知识点3：线性DP与背包**
 - 学习：0/1背包、完全背包、多重背包、LIS、LCS。
 - 刷题：P1060 (01背包), P1616 (完全背包), P2722 (完全背包), P1048 (01背包)。

第二阶段（Month 3-4）：数据结构算法突破

- **目标：** 攻克提高组最核心的数据结构。
- **知识点1：并查集**
 - 学习：路径压缩、按秩合并、带权并查集、种类并查集。
 - 刷题：P3367 (模板), P1551 (亲戚), P1196 (银河英雄传说), P1621 (集合)。
- **知识点2：树状数组与线段树**
 - 学习：单点修改+区间查询；区间修改+区间查询；离散化。
 - 刷题：P3374 (树状数组1), P3368 (树状数组2), P3372 (线段树1), P3373 (线段树2)。
- **知识点3：单调栈/队列与优化**
 - 你已有的“玩具装箱”和“Land Acquisition”就是最好的例子。重新用现代C++的思想重写一遍，确保理解为什么能用斜率优化，而不是只会套模板。
 - 刷题：P1886 (滑动窗口/单调队列), P6033 (合并果子/加强版-堆的变体)。

第三阶段 (Month 5-6)：提速与稳定，综合应用

- **目标：** 综合运用，提升解题速度。
- **方法：** 挑战CSP-S真题（近3年的真题）。不要只做一道，要一套一套做，严格限时。
- **推荐练习：**
 - **代码实现能力提升：** 练习将你的思想和代码翻译成高效代码。重点练习“构造题”和“大模拟题”。
 - **考前冲刺：** 洛谷的“月赛”和“提高组同步赛”是极好的练兵场。每周参与一次，感受比赛氛围。
 - **错题复盘：** 对于你过程中所有WA/TLE的题目，严格按照“四段式复盘法”进行操作（赛时模型、错因、正解性质、代码不变量）。

8. 【核心建议（优先级排序）】

- **🔴 紧急: 立即重构知识体系。** 不要再盲目刷题。花2天时间，对照CSP-S考纲，把每个知识点写在纸上，像拼地图一样，把缺失的部分用笔圈出来。你必须先知道自己“不知道什么”。
- **🔴 紧急: 强化数学能力。** 这是你最大的硬伤。每天至少抽30分钟专门学习数论和组合数学。从最简单的整除、素数开始。
- **🟡 重要: 改变编程习惯。** 停止使用 `auto` 和 `range-for` 的“抗拒”。从今天起，所有的循环都用 `for (int x : ve)` 来写。变量名必须保证在10行内能看懂。这将显著提升你的代码可维护性。
- **🟡 重要: 放弃“先难后易”的怪圈。** 你能写出斜率优化这种省选难度的DP，却不会写基础的01背包。这说明你学习的顺序是跳跃的、不扎实的。请回到第一阶段，踏实刷完背包和基础DP。
- **🟢 建议: 降低心理预期，享受解题过程。** 438道AC是一个非常棒的成就。但不要让它成为你的负担。接下来的目标不是“刷更多题”，而是“弄懂每道题”。多思考“为什么这么做”，而不是“怎么AC”。

- **建议: 学习使用** `std::array`, `std::tuple` 等现代C++语法, 并考虑使用 `constexpr` 和 `const` 来增加代码安全性。虽然比赛环境可能不支持所有新特性, 但学习它们可以极大提升你的抽象能力。

9.【未通过题目专属题解（从暴力到正解）】

很遗憾, 根据你提供的数据, “本次导出中未通过/卡住题数: 0”。这表明你似乎没有提供任何你做错或卡住的题目。

这本身就是最大的问题。

解读: 一名顶级选手必须有“卡住”的题目。正是因为没有任何记录, 我更确认了你前面提到的“复盘与错因沉淀缺失”。你很可能沉浸在AC的虚假繁荣中, 忽略了那些你“AC了但没搞懂”或“尝试失败就放弃”的题目。

为了完成这一部分, 我假设你正在尝试下面这道全国联赛常见题进行示范:

题目（虚拟示例）: —— 有一棵树, 求所有点对 (u, v) 的距离之和。 ($N \leq 2000$ 分)

a) AI 题解摘要: 本题的核心是计算每条边被经过的次数。一条边 (u, v) 会把树分成两部分, 大小为 `size_u` 和 `n - size_u`, 那么这条边的贡献就是 `边权 * size_u * (n - size_u)`。

b) 暴力思路: - **怎么做?** 对树上每个点进行DFS, 计算它到所有其他点的距离。所有距离累加。
- **复杂度:** $O(N^2)$ 。因为每个点需要 $O(N)$ 时间。 - **能拿部分分?** 可以拿30-50分（取决于数据范围）。 $N=2000$ 可以全拿满分。但对 $N=1e5$, 这是完全不可行的。

c) 瓶颈在哪里? - **时间瓶颈:** 重复计算。对每个点都进行遍历, 把整棵树遍历了 N 次, 做了大量无意义的重复工作。 - **空间瓶颈:** $N=2000$ 时没有空间问题。

d) 关键性质/不变量观察 - 关键观察: 这个问题的答案等价于 **每条边贡献的次数**。 - 想象一下, 一条边把树分成左右两部分。任何一条从一个部分到另一个部分的路径, 都必须经过这条边。 - 所以, 这条边被经过的次数 = 左边部分点的数量 * 右边部分点的数量。 - 我们需要的就是找到一种方法, 把 N 次遍历分解成一次DFS。

e) 最终正解的推导: 1. **一次DFS求子树大小:** 从任意根（比如1）开始做一次DFS。在遍历过程中, 计算出每个节点子树的大小 `sz[v]`。 2. **计算每条边的贡献:** 当我们递归到节点 `v` 时, 它到父节点 `u` 的边权为 `w`。那么这条边把树分成了大小为 `sz[v]` 和 `n - sz[v]` 的两部分。 3. **累加贡献:** 答案 += `w * sz[v] * (n - sz[v])`。 4. **核心代码结构（伪代码）:**

```
#include <bits/stdc++.h>
using namespace std;
const int N = 1e5 + 5;
```

```

vector<pair<int, int>> g[N]; // 邻接表存图 (节点, 边权)
int n, sz[N];
long long ans = 0;

void dfs(int u, int fa) {
    sz[u] = 1; // 包含自己
    for (auto &[v, w] : g[u]) {
        if (v == fa) continue;
        dfs(v, u);
        // 边权为 w, 子树大小为 sz[v]
        ans += 1LL * w * sz[v] * (n - sz[v]);
        sz[u] += sz[v];
    }
}

int main() {
    ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
    cin >> n;
    for (int i = 1, u, v, w; i < n; ++i) {
        cin >> u >> v >> w;
        g[u].emplace_back(v, w);
        g[v].emplace_back(u, w);
    }
    dfs(1, 0);
    cout << ans << '\n';
    return 0;
}

```

对比你的代码习惯： 注意我使用了 `auto &[v, w]` (这是C++17的structured binding)，取代你以前 `pii p=g[u][i]; int v=p.first;`，代码精简了很多。同时变量命名清晰 (`u, v, w, sz, ans`)，完全可读。

f) 推荐同类题： - **P4387 【模板】树的中心性：** 考察你能否自己推导出树上的“重心”性质，并计算重心到所有点的距离和。 - **推荐理由：** 该题的核心思想和本题完全一致，就是通过子树大小进行计算，是验证你理解程度的好题。 - **P1137 旅行计划：** 图上（或树上）基于拓扑排序/树形DP计算到达每个点的“路径数”或“和”。同样是利用了一次遍历统计答案的思想。 - **推荐理由：** 这道题加入了“图论”元素（有向无环图），可以帮助你理解“树上推导的结论，如何推广到图上”。

教练寄语：

438题是你的勋章，也是你的天花板。你已经在这个舒适区里待了很久了。现在是时候，砸碎它，去迎接那些会让你彻夜难眠、讨论不休的真正难题。

不要怕犯错，不要怕卡题。**你最怕的不是难题，而是永远在舒适区里刷简单题。** 今天，就从这里开始，重新出发。